13. Corrutinas en Unity

Las corrutinas permiten ejecutar procesos que se desarrollan a lo largo del tiempo **sin bloquear el juego**. Son ideales para animaciones temporizadas, recargas, disparos periódicos, comportamientos de enemigos, apariciones de objetos, etc.
Aunque se ejecutan dentro de un único hilo, Unity las va "pausando" y "retomando" gracias a la instrucción yield.

13.1. ¿Qué es una corrutina?

Una corrutina es una función que:

- Devuelve IEnumerator
- Puede suspender su ejecución usando yield
- Puede reanudarse en el siguiente frame o tras un periodo de tiempo

Ejemplo básico:

```
IEnumerator Ejemplo()
{
    print("Inicio");
    yield return new WaitForSeconds(2);
    print("Han pasado 2 segundos");
}
```

Para iniciarla:

```
StartCoroutine(Ejemplo());
```

13.2. Métodos más usados en yield

- `yield return null;`
  Espera **1 frame**.
- `yield return new WaitForSeconds(tiempo);`
  Espera **tiempo** segundos.
- `yield return new WaitUntil(condición);`
  Espera hasta que una condición se cumpla.
- `yield return new WaitForEndOfFrame();`
  Espera al final del frame.

13.3. Corrutinas en los scripts del proyecto

13.3.1. Enemigo – Disparo repetitivo

El enemigo dispara automáticamente cada cierto tiempo:

```
IEnumerator DisparoRepetitivo()
{
    while (true)
    {
        yield return new WaitForSeconds(tiempoDisparo);
        CrearBala();
    }
}
```

Se inicia en Start():

```
StartCoroutine(DisparoRepetitivo());
```

El uso de un **bucle infinito while (true)** es común cuando se necesitan acciones repetitivas durante toda la partida.

13.3.2. Enemigo – Parpadeo al recibir daño

Una corrutina se usa para cambiar temporalmente el color del enemigo al ser golpeado:

```
IEnumerator RecibirImpacto()
{
    spriteRenderer.color = new Color(0, 0.85f, 0.93f, 0.5f);
    yield return new WaitForSeconds(0.5f);
    spriteRenderer.color = Color.white;
}
```

Se activa dentro de OnTriggerEnter2D al recibir daño.

13.3.3. GameManager – Aparición de rocas

El GameManager genera rocas periódicamente:

```
IEnumerator SpawnRoca()
{
```

```
    while (true)
    {
        yield return new WaitForSeconds(Random.Range(2f,4f));
        CreaRoca();
    }
}
```

Esto crea un spawn de enemigos/objetos con tiempos variables gracias a Random.Range.

13.3.4. Humano – Sistema de recarga automática

Cada 2 segundos se intenta recargar una bala:

```
IEnumerator RecargaBala()
{
    while (true)
    {
        yield return new WaitForSeconds(2);
        if (municion < maxMunicion) municion++;
    }
}
```

Permite gestionar munición sin sobrecargar Update.

13.4. ¿Por qué usar corrutinas en lugar de Update()?

Mejor legibilidad

En vez de comprobar temporizadores manualmente:

timer -= Time.deltaTime;

if (timer <= 0) { ... }

Con una corrutina es más limpio:

yield return new WaitForSeconds(3);

Menos uso de CPU

Update se ejecuta **cada frame**, mientras que una corrutina puede esperar sin ejecutarse hasta que toque.

Permiten procesos secuenciales

Perfectas para animaciones, cooldowns, efectos temporales, cambios de estado, etc.

13.5. Cancelar corrutinas

Para detener una corrutina concreta:

```
StopCoroutine(nombreCorrutina);
```

O todas:

```
StopAllCoroutines();
```

Esto puede ser útil, por ejemplo, cuando el jugador muere o cambia de estado.

13.6. Buenas prácticas

- Usar corrutinas para tareas secuenciales o temporizadas
- Evitar grandes bucles dentro de corrutinas
- No abusar de StartCoroutine desde Update
- Para lógica de físicas usar FixedUpdate, no corrutinas
- Recordar siempre que WaitForSeconds se ve afectado por Time.timeScale